

vLLM 模型接入指南

从零开始：把你的模型搬上 vLLM 推理引擎

基础接入

张量并行

权重加载

多模态

ASR / 语音

AGENDA

全篇路线图

01

起步

vLLM 是什么 · 接入流程总览

P3 — P4

02

基础接入

引入代码 · prefix · forward · 权重加载 · 注册

P5 — P13

03

进阶能力

多模态接入 · 处理器与占位符

P14 — P16

04

语音识别

SupportsTranscription 接口

P17

05

测试 & FAQ

单元测试 · 滑动窗口 · Mamba · 总结

P18 — P20

WHAT IS vLLM

为什么要把模型接入 vLLM ？

高吞吐推理

PagedAttention 让 KV-Cache 利用率显著提高，单卡服务更多请求。

张量并行

内置 Row/Column/QKV ParallelLinear，几行替换即可拆到多卡。

统一接口

OpenAI 兼容的 /v1/chat、/v1/audio/transcriptions 等端点。

生态完善

已支持 Llama、Qwen、Whisper、LLaVA 等主流模型可参考。

接入意味着.....

- 把 HuggingFace 的 modeling_*.py 改写成 vLLM 风格
- 把 nn.Linear / nn.Embedding 替换为并行版本
- 实现 load_weights，把 HF checkpoint 装进来
- 在 ModelRegistry 注册架构名（内置或外部插件）
- （可选）多模态 / ASR / 滑动窗口 / Mamba 等能力

ROADMAP

接入流程：五步走



建议 workflow

① 在 `vllm / model_executor/models/` 中找一个最像你的模型（如 `llama.py`），先 `cp` 出一份再改。② 先跑通 `load_weights` 和单 GPU forward，再做并行。③ 用 `tests/models/registry.py` 注册测试用例，dummy weights 跑通即可合入。

STEP 01

引入模型代码

要点

1

从源仓库复制

clone HuggingFace 的 `modeling_*.py`，作为改写起点。

2

对照已有模型

参考 `vllm/model_executor/models/llama.py` 等已存在的实现。

3

保留协议许可

遵守原仓库的版权与许可（Apache-2.0 / MIT 等）。

4

移除训练代码

drop 掉 `loss`、`gradient checkpoint` 等推理不需要的部分。

● ● ● *shell · 引入模型代码*

1. 先找参考实现

```
$ ls vllm /model_executor/models/ | grep -i llama  
llama.py llama4.py llama_eagle.py
```

2. 复制 HF modeling 文件

```
$ wget https://github.com/huggingface/transformers/blob/main/src/transformers/models/llama/modeling_llama.py
```

3. 对照 vLLM 风格改写

```
$ cp modeling_llama.py vllm /model_executor/models/modeling_llama.py  
$ vim vllm /model_executor/models/modeling_llama.py
```

⚠ 注意版权与许可

务必检查并保留原始代码的 `license` 头部；vLLM 使用 Apache-2.0，不兼容的 `license` 不能直接合入主仓。

STEP 02 · 兼容 vLLM (1/3)

prefix 参数：每个模块的 " 身份证 "

prefix 是什么？

vLLM 要求所有内部模块在 `__init__` 里接收一个 `prefix` 字符串，通常等于该模块在 `state_dict` 中的全名。

1. 运行时支持

vLLM 的 `attention` 算子按全名注册，每个 `attention` 必须有唯一 `prefix` 避免冲突。

2. 量化支持

运行时根据 `prefix` 匹配量化配置，决定该层是否走量化路径，从而支持非均匀量化。

调用链示意

```
MyModelForCausalLM(prefix="")
└─ MyModel(prefix="model")
    └─ MyDecoderLayer(prefix="model.layers.0")
        └─ MyAttention(prefix="model.layers.0.self_attn")
            └─ Attention(prefix="... self_attn.attn") ★
└─ MyMLP(prefix="model.layers.0.mlp")
```

★ `Attention` 算子的 `prefix` 必须全局唯一，否则注册时冲突。

★ 命名通常用 `f'{prefix}.layers.{i}'` 与 HF 权重 `key` 对齐。

STEP 02 · 兼容 vLLM (2/3)

初始化代码：把 **prefix** 一路传下去

```
from torch import nn
from vllm.config import VllmConfig
from vllm.model_executor.layers.attention import Attention

class MyAttention(nn.Module):
    def __init__(self, vllm_config: VllmConfig, prefix: str):
        super().__init__()
        self.attn = Attention(prefix=f"{prefix}.attn")

class MyDecoderLayer(nn.Module):
    def __init__(self, vllm_config: VllmConfig, prefix: str):
        super().__init__()
        self.self_attn = MyAttention(prefix=f"{prefix}.self_attn")

class MyModel(nn.Module):
    def __init__(self, vllm_config: VllmConfig, prefix: str):
        super().__init__()
        self.layers = nn.ModuleList([
            MyDecoderLayer(vllm_config, prefix=f"{prefix}.layers.{i}")
            for i in range(vllm_config.model_config.hf_config.num_hidden_layers)
        ])

class MyModelForCausalLM(nn.Module):
    def __init__(self, vllm_config: VllmConfig, prefix: str = ""):
        super().__init__()
        self.model = MyModel(vllm_config, prefix=f"{prefix}.model")
```

关键约束

- 构造函数签名固定：(vllm_config, prefix)
- prefix 用 f-string 拼接子模块名
- 顶层模型默认 prefix=""
- 层名要与 HF state_dict 一致
- vllm_config 包含 ModelConfig / CacheConfig / ParallelConfig 等

STEP 02 · 兼容 vLLM (3/3)

重写 forward · 添加 embed_input_ids

① 添加 embed_input_ids

提供统一的文本嵌入入口；多模态组合模型也使用同一接口。

```
embed_input_ids

class MyModel(nn.Module):
    ...
    def embed_input_ids(
        self, input_ids: torch.Tensor
    ) -> torch.Tensor:
        return self.embed_tokens(input_ids)
```

② 重写 forward

input_ids 与 positions 都是扁平张量（无 max-seq 维），删除训练相关代码。

```
forward 签名

def forward(
    self,
    input_ids: torch.Tensor | None,
    positions: torch.Tensor,
    intermediate_tensors: IntermediateTensors | None = None,
    inputs_embeds: torch.Tensor | None = None,
) -> torch.Tensor:
```

HF vs vLLM forward 关键差异

维度	HuggingFace	vLLM
输入张量	(batch, seq_len)	(total_tokens,) 扁平
训练逻辑	保留 loss / labels	全部移除
Attention	自实现 SDPA / FlashAttn	调用 vllm .Attention 算子

STEP 03 (可选)

张量并行：把大模型拆到多卡

为什么需要张量并行？

当模型大到放不下单卡时，把权重沿行 / 列切分到多个 GPU，前向时通过 all-reduce / all-gather 同步结果。vLLM 提供开箱即用的并行层，几行替换即可启用。

替换思路

<code>nn.Embedding</code>	→ <code>VocabParallelEmbedding</code>
<code>nn.Linear (LM Head)</code>	→ <code>ParallelLMHead</code>
<code>nn.Linear (FFN -1)</code>	→ <code>MergedColumnParallelLinear</code>
<code>nn.Linear (Q / K / V)</code>	→ <code>QKVParallelLinear</code>
<code>nn.Linear (FFN -2 / O -Proj)</code>	→ <code>RowParallelLinear</code>



替换为并行层

```
from vllm.model_executor.layers.linear import (
    QKVParallelLinear, RowParallelLinear,
    MergedColumnParallelLinear,
)

class MyAttention(nn.Module):
    def __init__(self, config, prefix: str):
        super().__init__()
        # 一个调用同时产出 Q / K / V 三个分片
        self.qkv_proj = QKVParallelLinear(
            hidden_size= config.hidden_size,
            head_size= config.head_dim,
            total_num_heads= config.num_attention_heads,
            total_num_kv_heads= config.num_key_value_heads,
            prefix= f'{prefix}.qkv_proj',
        )
        self.o_proj = RowParallelLinear(
            input_size= config.hidden_size,
            output_size= config.hidden_size,
            prefix= f'{prefix}.o_proj',
        )
```

STEP 03 · 速查

并行线性层一览

类名	切分方式	通信操作	典型用途
ReplicatedLinear	权重复制，无切分	无	小层、不需要省显存的场合
ColumnParallelLinear	沿输出维（列）切分	无（输出按列分布）	FFN 第一层、Q/K/V 拆分
RowParallelLinear	沿输入维（行）切分	all-reduce（计算后）	FFN 第二层、Attn 输出 O-Proj
MergedColumnParallelLinear	多个 ColumnParallel 合并	无	SwiGLU 风格 FFN（gate + up）
QKVParallelLinear	Q/K/V 一起切分	无（含 KV-head 复制）	MHA / GQA 注意力的 QKV 投影
VocabParallelEmbedding	沿词表维切分	all-reduce	替代 nn.Embedding
ParallelLMHead	沿词表维切分	用于输出 logits	LM Head（输出投影）

TIP 所有并行 Linear 都支持 `linear_method` 参数，vLLM 会按量化方案自动注入；你不必手写量化逻辑。

STEP 04

实现 `load_weights` : 把 HF checkpoint 装进来

在 `*ForCausalLM` 里实现 `load_weights(weights)`。对于 Merged / QKV 这类合并层，要按原 HF 权重的多个 key 分块加载。

```
def load_weights(self, weights: Iterable[Tuple[str, torch.Tensor]]):
    # HF 中 QKV 是分开的; vLLM 用 QKVParallelLinear 合并
    stacked_params_mapping = [ # (vllm_param, hf_weight, shard_id)
        ("qkv_proj", "q_proj", "q"),
        ("qkv_proj", "k_proj", "k"),
        ("qkv_proj", "v_proj", "v"),
        ("gate_up_proj", "gate_proj", ),
        ("gate_up_proj", "up_proj", ),
    ]
    params_dict = dict(self.named_parameters())
    for name, loaded_weight in weights:
        for (param_name, hf_name, shard_id) in stacked_params_mapping:
            if hf_name not in name: continue
            name = name.replace(hf_name, param_name)
            param = params_dict[name]
            param.weight_loader(param, loaded_weight, shard_id) # 分块写入
            break
        else: # 普通参数 (无合并)
            param = params_dict[name]
            getattr(param, "weight_loader", default_weight_loader)(param, loaded_weight)
```

STEP 05

注册模型：让 vLLM 找到你的实现

内置 (Built-in)

合入 vLLM 主仓库

- Fork vLLM 仓库并从源码编译
- 把模型放进 `vllm/model_executor/models/`
- 在 `registry.py` 的 `_VLLM_MODELS` 中注册
- 保持字典字母序
- 在 `supported_models` 文档中加入

外部 (Out-of-tree)

通过插件加载

- 无需修改 vLLM 代码
- 在插件入口调用 `ModelRegistry.register_model`
- 若 `import` 会触发 CUDA 初始化，使用懒加载字符串形式
- 适合实验性 / 私有模型
- 多模态模型记得实现 `SupportsMultiModal`

STEP 05 · 代码

两种注册方式

方式 A · 内置注册

修改 registry.py 中的 _VLLM_MODELS 字典

```
registry.py

# vllm /model_executor/models/registry.py
_VLLM_MODELS: dict[str, BaseRegisteredModel] = {
    ...
    "MyModelForCausalLM": (
        "my_model",      # 文件名 (不含 .py )
        "MyModelForCausalLM", # 类名
    ),
    ...
}

# 字典按字母序维护
# 完成后即可:
# from vllm import LLM
# LLM(model="path/to/your-checkpoint")
```

方式 B · 插件注册

在外部包的入口函数 register() 中调用

```
my_plugin/__init__.py

# my_plugin/__init__.py
def register():
    from vllm import ModelRegistry

    # 直接注册 (如果模型 in port 不触发 CUDA )
    from your_code import YourModelForCausalLM
    ModelRegistry.register_model(
        "YourModelForCausalLM ",
        YourModelForCausalLM ,
    )

    # 或: 懒加载字符串形式 (避免 fork 时报
    # 'Cannot re-initialize CUDA in forked subprocess' )
    ModelRegistry.register_model(
        "YourModelForCausalLM ",
        "your_code:YourModelForCausalLM ",
    )
```

MULTIMODAL

多模态接入：在文本模型基础上扩展

①	更新基础模型	实现 <code>SupportsMultimodal</code> · 添加 <code>get_placeholder_str</code>
②	处理信息	<code>BaseProcessingInfo</code> · <code>get_supported_mm_limits</code>
③	Dummy 输入	<code>BaseDummyInputsBuilder</code> · <code>get_dummy_text/data</code>
④	处理细节	<code>BaseMultimodalProcessor</code> · 字段配置 · <code>prompt</code> 更新
⑤	注册装饰器	<code>@MULTIMODAL_REGISTRY.register_processor</code>

推荐参考实现：LLaVA（图像） · Qwen3-Omni（音频 + 图像） · Voxtral（音频） · Fuyu（特殊占位）

MULTIMODAL · 模型层

embed_multimodal 与组件标记

把 HF 原本写在 forward 里的视觉编码 + 投影逻辑，搬到 embed_multimodal；文本嵌入与合并由 vLLM 默认实现自动处理。

● ● ● your_model.py · 多模态结构

```
from vllm.model_executor.models.interfaces import SupportsMultimodal
from vllm.multimodal.inputs import MultimodalEmbeddings

class YourModelForImage2Seq(nn.Module, SupportsMultimodal):
    @classmethod
    def get_placeholder_str(cls, modality, i):
        return "<image>" if modality.startswith("image") else None

    def __init__(self, *, vllm_config, prefix=""):
        super().__init__()
        config = vllm_config.model_config.hf_config
        with self.mark_tower_model(vllm_config, "image"): # 视觉塔
            self.vision_encoder = ...; self.multimodal_projector = ...
        with self.mark_language_model(vllm_config): # 语言塔
            self.language_model = init_vllm_registered_model(
                vllm_config=vllm_config, hf_config=config.text_config,
                prefix=maybe_prefix(prefix, "language_model"))

    def embed_multimodal(self, **kwargs) -> MultimodalEmbeddings | None:
        image_input = self.parse_and_validate_image_input(**kwargs)
        if image_input is None: return None
        return self.multimodal_projector(self.vision_encoder(image_input))
```

MULTIMODAL · 处理器

占位符替换与字段配置

处理器三件套

BaseProcessingInfo

`get_supported_mm_limits()`
声明每个模态的最大输入数

BaseDummyInputsBuilder

`get_dummy_text()` / `get_dummy_mm_data()`
用于显存预估，构造最坏情况输入

BaseMultiModalProcessor

`_get_mm_fields_config()`
`_get_prompt_updates()`
描述张量形状与占位符替换规则



注册多模态处理器

```
from vllm.model_executor.models.interfaces import \
    SupportsMultiModal
from vllm.multimodal import MULTIMODAL_REGISTRY

@MULTIMODAL_REGISTRY.register_processor(
    YourMultiModalProcessor,
    info=YourProcessingInfo,
    dummy_inputs=YourDummyInputsBuilder,
)
class YourModelForImage2Seq(
    nn.Module, SupportsMultiModal
):
    ...

# _get_prompt_updates 中常用:
PromptReplacement(
    modality="image",
    target=[image_token_id],
    replacement=get_replacement, # 重复 N 次
)

# 也可用 PromptInsertion (不替换, 仅插入)
```


SPEECH-TO-TEXT

实现 SupportsTranscription 接口

必须实现

supported_languages

ISO 639-1 → 语言名 的映射表

supports_transcription_only

True 则模型不做文本生成（如 Whisper）

get_speech_to_text_config

采样率 / 最大音频时长 / 切分窗口

get_generation_prompt

构造提示：多模态字典 或 encoder/decoder 字典

```
asr_model.py

from typing import ClassVar, Mapping, Literal
from vllm.config import SpeechToTextConfig
from vllm.model_executor.models.interfaces import SupportsTranscription

class YourASRModel(nn.Module, SupportsTranscription):
    supported_languages: ClassVar[Mapping[str, str]] = {
        "en": "English", "zh": "Chinese", ...
    }
    supports_transcription_only: ClassVar[bool] = True

    @classmethod
    def get_speech_to_text_config(
        cls, model_config, task_type,
    ) -> SpeechToTextConfig:
        return SpeechToTextConfig(
            sample_rate=,
            max_audio_clip_s=,
            min_energy_split_window_size=None,
        )

    @classmethod
    def get_generation_prompt(cls, stt_params):
        return { ... } # 返回 PromptType
```

TESTING

单元测试：合入主仓的最低门槛

REQUIRED

必需测试

模型加载

在 `tests/models/registry.py` 添加 HF 仓库示例，CI 用 dummy weights 校验初始化

RECOMMENDED

推荐测试

生成模型对照

`check_outputs_equal` —— 与 HF 的文本完全一致

Logprobs 相似

`check_logprobs_close` —— Top-K 对齐

Pooling 模型

余弦相似度比较

OPTIONAL

多模态测试

通用处理

`tests/models/multimodal/processing/test_common.py`

模型专属

可在同目录下新增文件，验证关键字参数等行为

FAQ

常见问题：滑动窗口 · Mamba

Q1 · 交错滑动窗口

- ① 检查 `config.json` 中是否有 `layer_types` 字段。
- ② 在建模代码里逐层解析滑动窗口大小，传入 Attention 的 `per_layer_sliding_window` 参数。
- ③ 参考实现：`vllm / model_executor/models/lam_a.py` 中第 ~171 行。

Q2 · Mamba & Mamba-like

1 纯 Mamba

继承 `IsAttentionFree` · 用 `MambaMixer / MambaMixer2` · 加入 `MODELS_CONFIG_MAP`

2 Mamba + Attention

继承 `IsHybrid` · 实现 `get_mamba_state_*` · 参考 `Jamba / Bamba`

3 类 Mamba 自定义

继承 `MambaBase` · 实现 `get_state_dtype/shape/attn_backend` · 必要时注册 `custom op`

一图回顾，开始动手

- 01 引入代码 `找参考 / cp 改写`
- 02 兼容 **vLLM** `prefix · forward · embed_input_ids`
- 03 并行层 `QKV / Row / Column / Vocab Embedding`
- 04 权重加载 `load_weights + stacked mapping`
- 05 注册 `_VLLM_MODELS 或 plugin register()`

资源

官方文档

docs.vllm.ai/en/latest/

示例模型

vllm.ai/model_executor/models/

注册中心

model_executor/models/registry.py

测试入口

tests/models/registry.py

插件机制

docs.vllm.ai/.../plugin_system/

Happy Hacking with vLLM → 欢迎提 PR / Issue